

- Crossplane has been implemented in your Kubernetes cluster
 - Azure account with the necessary permissions
- Now, let's start with the configuration and deployment steps.

Step 1: Install Crossplane Azure provider

To commence the deployment of a web application on Azure using Crossplane, it is necessary to install the latter on your Kubernetes cluster. The following example illustrates the process of developing a web application that is connected to a PostgreSQL database backend and deployed using Crossplane orchestration. Install the Azure provider for Crossplane first. This provider enables Crossplane to oversee Azure resources.

```
run the following in bash prompt
kubectl crossplane install provider crossplane/provider-
azure:v0.17.0
```

Step 2: Create Azure provider config

The subsequent stage is to configure the Azure provider once Crossplane is operational. This allows Crossplane to oversee Azure platform resources. Generate a Kubernetes secret to house your Azure credentials. Establish a provider configuration for Azure that incorporates your Azure credentials.

```
```yaml script for Azure Provider config
apiVersion: v1
kind: Secret
metadata:
 name: azure-creds
namespace: crossplane-system
data:
 credentials:

apiVersion: azure.crossplane.io/v1alpha3
kind: ProviderConfig
metadata:
 name: example
spec:
 credentials:
 source: Secret
 secretRef:
 namespace: crossplane-system
 name: azure-creds
 key: credentials
  ```
```

Step 3: Define a resource class

The resources necessary for your database instance can be defined after the Azure provider has been configured. Begin by establishing a resource group and specifying a resource

class for the Azure PostgreSQL instance.

```
```yaml script to define resource class
apiVersion: database.azure.crossplane.io/v1beta1
kind: PostgreSQLServerClass
metadata:
 name: standard-postgres
specTemplate:
 location: West US
 resourceGroupName: my-group
 sku:
 tier: GeneralPurpose
 family: Gen5
 capacity: 2
 storageProfile:
 storageMB: 5120
 administratorLogin: myadmin
 administratorLoginPasswordSecretRef:
 name: db-password
 key: password
  ```
```

Step 4: Create managed resources

Establish a managed resource for the PostgreSQL instance like the yaml script given below:

```
```yaml script for managed resource config to PostgreSQL
database instance
apiVersion: database.azure.crossplane.io/v1beta1
kind: PostgreSQLServer
metadata:
 name: my-postgres
spec:
 forProvider:
 resourceGroupName: my-group
 location: West US
 sku:
 tier: GeneralPurpose
 family: Gen5
 capacity: 2
 storageProfile:
 storageMB: 5120
 administratorLogin: myadmin
 administratorLoginPasswordSecretRef:
 name: db-password
 key: password
 providerConfigRef:
 name: example
  ```
```

Step 5: Deploy the web application

Once the AKS cluster is operational, it is possible to deploy your web application. Begin by establishing a Kubernetes deployment and service for the application. Deploy the web